GINJA

Live Rig Monitor

Product Release Handbook

From Studio To Stage



Master Your Tools.
Monitor Your Rig.
Find Your Creative Flow.



Created By Ry Dos

Documentation Preview / October 2026

PRODUCT DELIVERY

Release Scope And Audience

01 / The Current Deliverable

The current public deliverable is a portfolio documentation release: visual manual, product explanation, architecture, support and roadmap. Source and executable stay private. The creator reports practical use in their own rig. The repository does not announce a tested installer, customer deployment or verified compatibility with every represented device.

02 / The Interviewer Journey

The README introduces the problem, shows the dashboard and links to the engineering story. A reviewer can then inspect the visual manual, configuration exercises and validation plan. These layers answer different questions: what was built, why it was built, how to use it and what evidence still needs collection.

03 / A Future Application Release

A downloadable release requires a resolved dependency-license route, a packaged build, second-PC acceptance results, versioned release notes, required notices and a support path. Preserve the public showcase while those items are completed. Documentation polish alone does not convert a private prototype into a validated distributable product.

PRODUCT DELIVERY

What Belongs In The Repository

01 / Upload The Public Folder

Extract the Upload To GitHub ZIP and place README.md at the repository root. Keep docs/assets, the PDF manuals, supporting Markdown and .github issue templates in their existing folders. Relative links depend on those paths. Do not flatten the assets or upload only the ZIP as the repository content.

02 / Keep Private Material Separate

The separate private review, résumé handoff, build kit, source files, EXEs, raw logs and account credentials do not belong in this public upload package. The profile kit is also separate: its README belongs to the personal profile repository, not the product repository.

03 / Confirm The Published Result

Open the repository as a visitor and scroll through its README. Click the manual, case study and roadmap links, check image rendering and verify availability wording. Add the final repository URL to your résumé only after this check. A product website is optional; a clear README can serve as the portfolio entry point.

PRODUCT DELIVERY

Prerequisites And Installation

01 / State The Requirements

Document the Windows versions actually tested, architecture, device drivers and external applications. MIDI hardware and an audio interface require their own setup. Ableton, OBS, Discord and PresentMon are separate products. Do not invent a minimum CPU/RAM specification without testing the intended workload.

02 / Portable Installation

For the current private portable approach, extract the complete package, preserving the executable support folders and audio-discovery helper. Test on a PC without Python or development tools. A standalone-looking EXE may still require the other files beside it. An installer can be introduced later with separately tested install/update/remove behavior.

03 / First-Run Acceptance

Confirm dashboard/artwork loading, driver access, one known MIDI input and application detection. Configure OBS credentials deliberately and start with a simple workspace. Record the environment and result. Failure should lead to a small, reproducible report rather than a broad unsupported claim about the user's hardware.

PRODUCT DELIVERY

Configuration And Persistence

01 / Configuration Areas

Appearance, dashboard visibility, device links, OBS connection, monitoring/startup and system-overlay selections serve different purposes. Routing rules define explicit triggers/actions. Workspace profiles preserve layout/appearance and watched apps, not every application setting. Explain these boundaries so users do not treat a profile as a complete backup.

02 / Saved And Temporary State

The application uses Qt settings under the Ginja / LiveRigMonitor identity. The reviewed OBS implementation saves host/port and keeps the password only for the current run. Discord profile connection and public activity sharing are separate choices. Saved or migrated preferences can restore values after restarting.

03 / Migration And Recovery

Before changing versions, preserve the working build and document settings/rules. A universal configuration export/import or reset workflow has not been documented as an implemented feature. Introduce a supported backup/migration procedure only after testing it. Avoid destructive reset instructions as a first response to a symptom.

PRODUCT DELIVERY

Versioning And Release Notes

01 / Choose A Meaningful Version

Set a software version when a build is actually prepared. Major changes can signal incompatible behavior, minor changes added functionality and patch changes fixes. The documentation revision V3 is not an application version or proof that an installer exists. Keep those two histories distinct.

02 / Describe The User Impact

For each release, state the concrete problem and resulting behavior. Include added/changed/fixed items, known limitations, tested environments and any configuration migration. Avoid claiming generic stability improvements without evidence. Future features belong in the roadmap until implemented and tested.

03 / Preserve The Record

Retain the reviewed package, checksum, dependency inventory and acceptance report for the released version. Compare changes with a known baseline. If a package is replaced, give it a new identifiable version and explain why, rather than silently replacing a file reviewers may already have downloaded.

PRODUCT DELIVERY

Packaging And Dependency Inventory

01 / Define The Package

Include the application runtime, required artwork/tutorial assets and any packaged helper needed for audio enumeration. Exclude private settings, recordings, temporary work files and source where the chosen license route permits it. Keep external applications separate and clearly identify which components users install themselves.

02 / Inventory What Is Bundled

Record exact versions and applicable licenses for Python, the Qt binding/runtime, MIDI, audio, system and optional connection libraries. Also review native libraries included indirectly by the packager. A requirements file alone is not a complete inventory of the files inside an EXE package.

03 / Validate The Result

Build in a controlled environment and test without development tools. Check artwork paths, the audio helper, startup/tray behavior, MIDI port access and external connections. A build script passing syntax checks is not equivalent to a successful binary build or target-machine test.

PRODUCT DELIVERY

Licenses, Attribution And Media

01 / Separate Presentation From Distribution

Keep the showcase independent of the unresolved executable-distribution decision. The current app imports PyQt6; proprietary binary distribution requires an appropriate licensing route. A commercial entitlement or a validated alternative such as a suitable PySide/Qt route requires deliberate review. A disclaimer cannot replace those obligations.

02 / Check Public Visual Assets

Establish permission for photos, recordings, friend artwork and third-party imagery before publication. Product names identify the tools discussed; no endorsement or certification is claimed. Identify screen replacements and illustrative activity. Do not treat an animated meter or concept integration as evidence of a delivered live connection.

03 / Keep Rights Evidence

Retain Suno subscription/track records and the original human-created contribution evidence. Paid-plan commercial-use rights do not guarantee copyright protection. Keep the owner identity and dependency rights separate. Review final usage terms and included notices before offering a downloadable executable.

PRODUCT DELIVERY

Privacy And Security Boundaries

01 / Document Data Handling

Explain local preferences, event history and session reports, including that reports may contain names, paths and errors. Document OBS endpoint use, optional ping, MIDI output ports and Discord profile/activity choices. Avoid promising no network use when optional connections are configured.

02 / Report Responsibly

Public Issues are for reproducible, redacted bug reports. Never post passwords, user tokens, private account details or unrestricted logs there. A dedicated confidential security contact has not been established; define and verify one before inviting sensitive reports. Do not imply that the GitHub username is a private disclosure channel.

03 / Avoid Unsupported Assurance

No independent security audit or broad device certification is asserted. A future release needs threat/data-flow review, sensitive-data handling checks and a documented response process appropriate to its scope. Distinguish read-only observation from MIDI/OBS actions that change external state.

PRODUCT DELIVERY

Acceptance Testing And Evidence

01 / Use A Reproducible Matrix

Test launch, artwork, MIDI input, trigger matching, ordered outputs, note pairing, OBS state/actions, popouts, profiles, reports, audio discovery and optional FPS capture. Record Windows, app/driver versions, devices and exact steps. Include unplug/reconnect and restart cases, not only the ideal path.

02 / Record Pass, Fail And Not Run

Keep the expected result, observed result and evidence reference for each case. Mark unavailable providers separately from zero readings. Publish only measured outcomes. The documentation review and link/layout checks performed here are distinct from runtime acceptance, which still needs target-PC execution.

03 / Manage Regressions

After a change, rerun affected workflows and a small baseline. If changing the Qt binding or packaging, broaden checks to UI, signals, helpers and saved settings. Do not invent test counts or badges. A concise honest test report is stronger than an unsupported production-ready label.

PRODUCT DELIVERY

Updates, Rollback And Removal

01 / Introduce Updates Deliberately

No automatic updater is claimed. A future portable update should go into a new folder so the old known-working package is retained. Recheck preference compatibility and required assets/helpers. Explain migrations in release notes and verify the result before a critical performance.

02 / Keep A Recovery Path

Preserve the prior build and a documented configuration baseline. If the new build fails, stop testing that build and return to the known setup where appropriate. Application settings may be shared between versions, so an older executable alone is not a guarantee of complete rollback.

03 / Document Removal

For a portable package, exit the application before removing its extracted folder. Keep wanted reports and configuration evidence first. Saved settings may remain outside that folder. Provide exact settings/data cleanup instructions only after identifying and testing those locations; do not recommend broad registry deletion.

PRODUCT DELIVERY

Support And Issue Triage

01 / Collect A Useful Report

Ask for version/build, Windows, relevant drivers/apps, expected behavior, actual behavior and minimal steps. Attach a redacted event excerpt or session report when helpful. State whether the issue occurs in the developer environment or a packaged build; those paths can behave differently.

02 / Keep The Scope Clear

The repository's issue templates provide a consistent starting point for bugs and suggestions. No response-time commitment, commercial support contract or compatibility guarantee is offered. Distinguish a Ginja defect from a driver, external application, hardware wiring or provider limitation.

03 / Close The Loop

Reproduce the issue, record the proposed fix and validate the same steps again. Update known limitations and release notes when appropriate. Tag a change as fixed only when evidence supports it. Feature requests should identify the workflow/problem rather than promising an integration date before investigation.

PRODUCT DELIVERY

Roadmap And Integration Design

01 / Build On Current Behavior

The present foundation is focused monitoring, popouts, diagnostics and explicit CC-triggered action sequences. It is expandable, but the current outputs remain MIDI and supported OBS actions. Broader applications of the pattern should be described as potential workflows rather than implemented arbitrary-protocol routing.

02 / Describe Upcoming Work

Push 3 extensions, deeper Ableton integration, Synesthesia coordination, Stream Deck and controller-input workflows are planned directions. For each, define the proposed input, expected action, user problem, dependencies and validation before announcing delivery. Keep screenshots labeled as concepts where appropriate.

03 / Show Progress Honestly

A roadmap can distinguish exploration, implementation, validation and delivered behavior once those states are established. Do not assign progress percentages or dates without evidence. Promote a capability into the current list only after implementing it and recording its exact supported scope.

PRODUCT DELIVERY

GitHub Presentation And Media

01 / Build The Landing Page

GitHub automatically renders README.md beneath the file list. Use a strong hero, concrete product statement, dashboard image and concise paths to the case study/manual. Put the most useful explanation first. A reviewer should not need to browse a maze of folders to understand the project.

02 / Make The Video Accessible

A linked video or separate uploaded video asset is not automatically a slideshow embedded in the README. Add a clear thumbnail and link once a final public URL is known. Repository assets must respect GitHub upload limits. A website through GitHub Pages is optional and must be configured separately.

03 / Present The Person And Project

The product repository and personal profile README have different roles. Pin Ginja to the profile and use the product link on the résumé. Use Ry Dos as the public creative identity, with employment/application naming handled consistently. Retain honest boundaries for ownership, AI-assisted work and measured results.

PRODUCT DELIVERY

The Release Decision

01 / Ready For Portfolio Review

The package provides the designed user manual, detailed workflows, case study, architecture, attribution/privacy, roadmap, support templates and release planning. It can be reviewed as a portfolio artifact. Final publication still requires asset-rights/privacy checks and verifying the uploaded page renders correctly.

02 / Before Downloads

Resolve distribution licensing, complete the build, run second-PC acceptance and supply usage terms/notices, accurate requirements, versioned release notes and checksums. Those are the remaining concrete delivery gates. An upload button or private repository does not waive dependencies' distribution obligations.

03 / Keep The Story Defensible

Present Ginja as a solution built around real workflow friction, with documented implementation and creator-reported use. Explain decisions and limitations in plain language. Do not invent customers, benchmark gains, employment experience or completed tests. Let the interface, tutorials and evidence carry the technical story.